RetailStream Data Platform

Architecture & Solution Design Document

---

1. Project Overview

Project Name

RetailStream – Enterprise Batch Data Platform

Objective

RetailStream is a production-inspired data engineering platform that simulates the data ecosystem of a large retail company. The platform collects data from multiple operational systems, processes it through a Medallion Architecture (Bronze, Silver, Gold), stores curated datasets in a data warehouse, and provides business intelligence dashboards and operational monitoring.

The goal of this project is to demonstrate how a modern data engineering platform is designed and implemented using open-source technologies while following enterprise software engineering practices.

---

2. What the Project Does

RetailStream simulates the complete data lifecycle of an e-commerce company.

The platform performs the following functions:

Data Generation

- Generates realistic retail datasets containing millions of records.
- Produces customers, products, suppliers, inventory, orders, and order items.
- Simulates business events over multiple years.
- Introduces configurable data quality issues for testing.

Source Systems

- Exposes operational data through REST APIs.
- Provides batch files (CSV) for systems that export data.
- Mimics the behavior of real production source systems.

Data Ingestion

- Uses Apache Airflow to orchestrate scheduled batch pipelines.
- Extracts data from APIs and files.

• Stores raw data in the Bronze layer without modification.

Data Processing

• Uses Apache Spark to clean, validate, standardize, and transform raw datasets.
• Builds curated Silver datasets.
• Creates Gold datasets optimized for business analytics.

Data Warehouse

• Loads business-ready datasets into PostgreSQL.
• Implements a dimensional model with fact and dimension tables.
• Supports historical reporting and analytical queries.

Business Intelligence

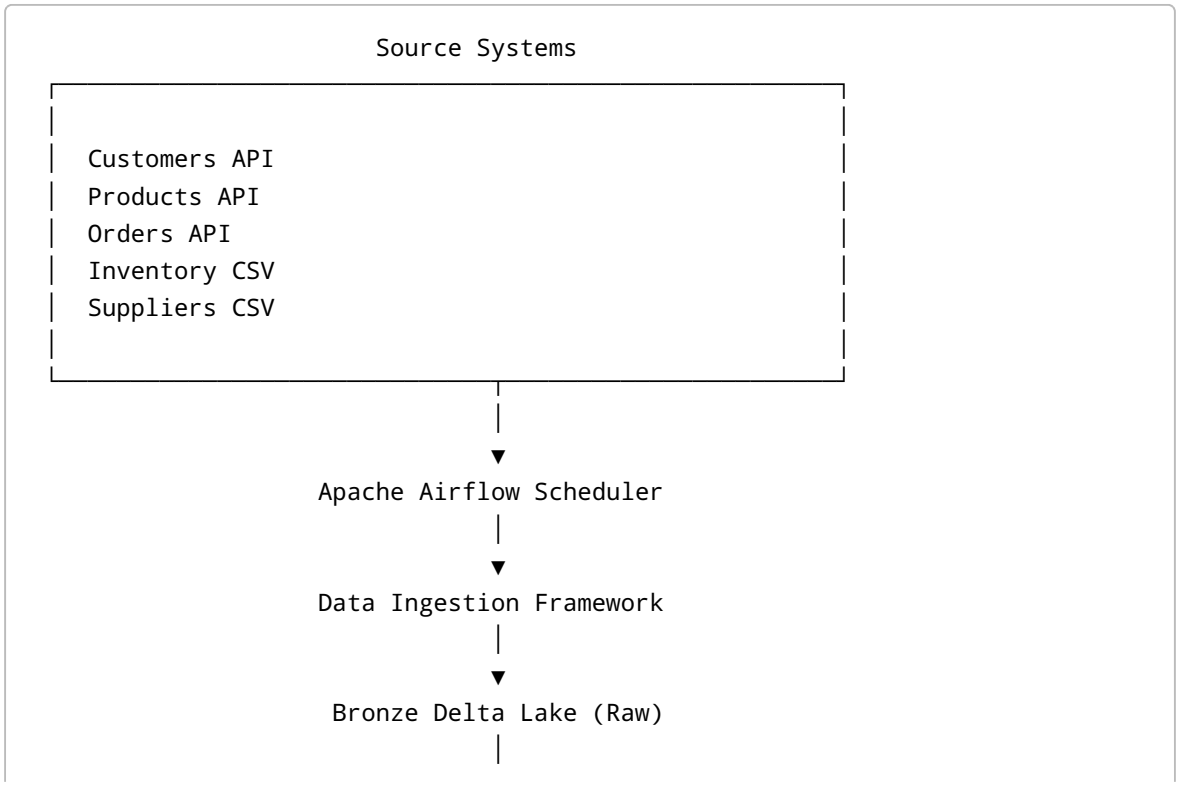
• Uses dbt for SQL transformations and business models.
• Creates reporting views.
• Visualizes KPIs using Apache Superset dashboards.

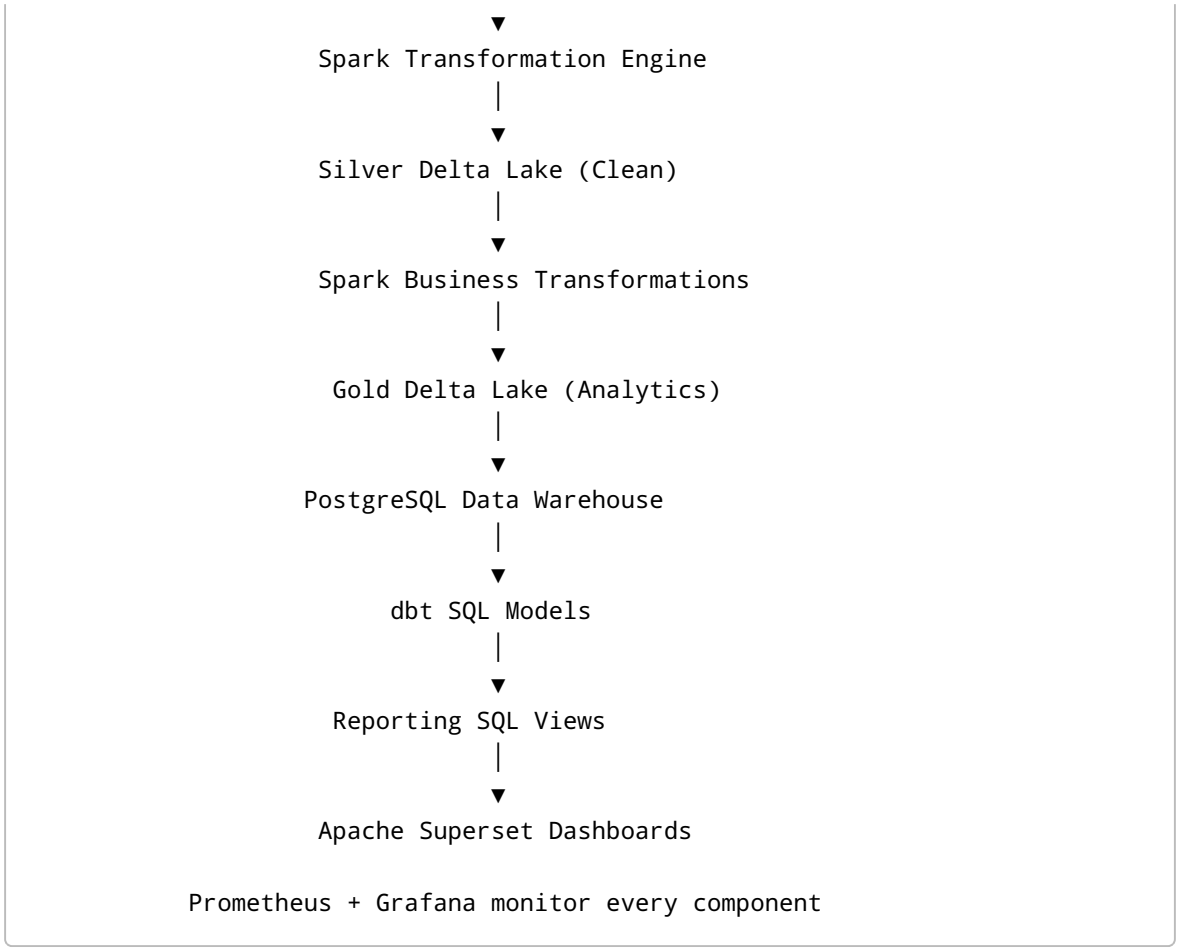
Monitoring

• Tracks pipeline execution.
• Monitors infrastructure health.
• Captures audit logs and operational metrics.
• Displays system health through Grafana dashboards.

---

3. High-Level Project Architecture





---

4. Technologies Used

Technology	Purpose
Python	Primary programming language
Apache Airflow	Workflow orchestration and scheduling
Apache Spark (PySpark)	Large-scale data processing and transformations
Delta Lake	Reliable storage format supporting ACID transactions
MinIO	Object storage for Bronze, Silver, and Gold datasets
PostgreSQL	Analytical data warehouse
dbt	SQL modeling, testing, and documentation
FastAPI	Simulated operational source systems
Apache Superset	Business intelligence dashboards
Prometheus	Metrics collection
Grafana	Infrastructure and pipeline monitoring

Technology	Purpose
Docker	Containerized deployment
GitHub Actions	Continuous Integration (CI)
PyTest	Unit and integration testing

---

5. How Each Technology Is Used

Python

Python acts as the foundation of the platform.

It is used for:

- Data generation
- API development
- Spark applications
- Airflow tasks
- Utility frameworks
- Configuration management
- Validation logic

FastAPI

FastAPI simulates operational business systems.

Examples:

- Customer Management System
- Product Catalog
- Order Management System

Airflow extracts data from these APIs on a schedule.

Apache Airflow

Airflow orchestrates every pipeline in the platform.

Responsibilities include:

- Scheduling daily jobs
- Managing dependencies
- Retrying failed tasks
- Monitoring executions

- Triggering Spark jobs
- Loading warehouse tables

Airflow is the control layer of the platform.

Apache Spark (PySpark)

Spark performs all large-scale data processing.

Examples:

- Reading raw files
- Cleaning data
- Removing duplicates
- Applying business rules
- Joining datasets
- Creating Gold datasets
- Incremental processing

Spark is the processing engine.

MinIO

MinIO provides S3-compatible object storage.

It stores:

- Bronze data
- Silver data
- Gold data
- Generated datasets
- Historical files

It serves as the project's data lake.

Delta Lake

Delta Lake sits on top of MinIO.

Benefits include:

- ACID transactions
- Schema evolution
- Time travel
- Incremental updates
- Reliable writes

Every data lake layer is stored using Delta format.

PostgreSQL

PostgreSQL acts as the analytical warehouse.

It contains:

- Fact tables
- Dimension tables
- Reporting views
- Business-ready datasets

Superset connects directly to PostgreSQL.

---

dbt

dbt manages SQL transformations after warehouse loading.

Responsibilities include:

- Creating staging models
- Building dimensional models
- Creating marts
- Running data quality tests
- Generating documentation

Apache Superset

Superset provides business dashboards.

Dashboards include:

- Executive Dashboard
- Sales Dashboard
- Customer Dashboard
- Inventory Dashboard
- Product Dashboard

Prometheus

Prometheus collects metrics from:

- Airflow

- Spark
- PostgreSQL
- Docker
- Custom pipeline metrics

Grafana

Grafana visualizes operational metrics.

Examples:

- Pipeline duration
- Success rate
- Records processed
- Failed jobs
- Infrastructure health

Docker

Docker runs every component in isolated containers.

This ensures:

- Easy deployment
- Consistent environments
- Simple local setup

GitHub Actions

GitHub Actions automates:

- Unit testing
- Code quality checks
- Docker builds
- Continuous Integration

---

6. Data Flow

The platform follows a layered Medallion Architecture.

Step 1 – Data Generation

A configurable generator creates realistic retail datasets with millions of records.

Outputs include:

- Customers
- Products
- Suppliers
- Inventory
- Orders
- Order Items

Step 2 – Source Systems

Generated data is exposed through:

- REST APIs
- CSV files

These simulate real production systems.

Step 3 – Data Ingestion

Airflow executes scheduled pipelines.

It:

- Calls APIs
- Reads CSV files
- Tracks execution
- Logs metadata

Step 4 – Bronze Layer

Raw data is stored exactly as received.

Characteristics:

- Immutable
- No transformations
- Complete history
- Audit metadata

Step 5 – Silver Layer

Spark cleans and validates data.

Typical transformations:

- Remove duplicates
- Standardize dates
- Validate email addresses
- Handle missing values
- Normalize categories
- Reject invalid records

Step 6 – Gold Layer

Spark creates business-ready datasets.

Examples:

- Daily Sales
- Monthly Sales
- Customer KPIs
- Product Performance
- Inventory Metrics

Step 7 – Warehouse

Gold datasets are loaded into PostgreSQL.

The warehouse contains:

- Fact tables
- Dimension tables
- Historical business data

Step 8 – SQL Modeling

dbt creates:

- Business models
- Analytical marts
- Reporting views
- Data quality tests

Step 9 – Business Intelligence

Apache Superset connects to PostgreSQL.

Business users access dashboards to monitor:

- Revenue
- Sales trends
- Customer behavior
- Inventory
- Product performance

Step 10 – Monitoring

Prometheus continuously collects metrics.

Grafana displays:

- Pipeline execution status
- Infrastructure health
- Data quality statistics
- Resource utilization

---

7. End-to-End Data Flow

```
Enterprise Data Generator
          │
          ▼
 REST APIs + CSV Sources
          │
          ▼
   Apache Airflow DAGs
          │
          ▼
   Bronze (Delta on MinIO)
          │
          ▼
   Silver (Spark Cleaning)
          │
          ▼
 Gold (Business Aggregation)
          │
          ▼
 PostgreSQL Warehouse
          │
          ▼
      dbt Models
          │
          ▼
 SQL Reporting Views
```

```
          |
          ▼
 Apache Superset Dashboards

_____________________________________________

Prometheus
      |
      ▼
Grafana Monitoring Dashboards
```

---

8. Expected Outcomes

After completion, the platform will demonstrate:

- Enterprise-scale batch data processing
- Modern Medallion Architecture
- Automated workflow orchestration
- Configurable data generation
- Robust data quality validation
- Dimensional data warehouse design
- Business intelligence reporting
- Operational monitoring
- CI/CD and testing
- Production-style project organization

This project is designed to resemble the architecture and engineering practices used in modern data teams, making it suitable as a portfolio project for Data Engineering roles.